

Day 6 — Power Management

Low Power Design Techniques for Embedded Systems

1 | Why Power Management Matters

Battery-powered and energy-harvesting devices live or die by their current budget. A typical ARM Cortex-M MCU draws ~10–50 mA active but only 1–10 μ A in deep sleep — a 1000 $\times$ difference. Proper power management extends battery life from days to years.

Active current	10–100 mA — CPU running, peripherals on, radio transmitting.
Sleep current	1–100 μ A — CPU halted, clocks gated, RAM retained.
Deep sleep / stop	100 nA–10 μ A — most clocks off, only RTC or LPUART running.
Shutdown / standby	10–300 nA — almost everything off; only backup domain alive.
Energy harvesting	Solar, RF, vibration: budget often <10 μ A average current.

2 | MCU Power Modes (STM32 Example)

Mode	CPU	Clocks	RAM	Wake Sources	Typical Current
Run	On	All on	Yes	—	10–50 mA
Sleep	Off	AHB/APB running	Yes	Any interrupt	1–5 mA
Low-Power Sleep	Off	MSI slow clock	Yes	Any interrupt	200–500 μ A
Stop 0/1	Off	Gated; LSI/LSE on	Yes	EXTI, LPUART, I2C, RTC	10–300 μ A
Stop 2	Off	Only LSI/LSE	Yes	EXTI, LPUART, RTC	1–10 μ A
Standby	Off	Off	No	WKUP pin, RTC, NRST	300 nA–2 μ A
Shutdown	Off	Off	No	WKUP pin, NRST	30–100 nA

On wakeup from Stop/Standby the MCU re-runs startup code and must restore clocks and peripheral state. Use backup registers (RTC_BKPxR) to preserve context.

3 | Entering Sleep Modes — STM32 HAL / LL

Sleep (WFI — Wait For Interrupt):

```
HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
```

Stop 2 mode:

```
HAL_PWREx_EnterSTOP2Mode(PWR_STOPENTRY_WFI);
```

Standby mode:

```
HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1);
```

```
HAL_PWR_EnterSTANDBYMode();
```

Re-initialise clocks after Stop wakeup:

```
SystemClock_Config(); // re-run your clock init function
```

■ *Disable all unused GPIOs and peripherals before entering Stop/Standby — floating pins and enabled peripherals are the #1 cause of unexpectedly high sleep current.*

4 | Clock Gating & Dynamic Frequency Scaling

Dynamic power ($P = C \cdot V^2 \cdot f$) scales linearly with frequency and quadratically with voltage. Run at the lowest clock that meets your real-time deadline.

Clock gating	Disable peripheral clocks when idle: <code>__HAL_RCC_TIM2_CLK_DISABLE()</code> .
Freq. scaling	Switch to MSI 4 MHz for light tasks; ramp to PLL 80 MHz for bursts.
Voltage scaling	STM32: <code>PWR_REGULATOR_VOLTAGE_SCALE2</code> lowers core voltage at lower freq.
PLL off in stop	PLL consumes ~1 mA; always turns off entering Stop mode automatically.
AHB/APB prescalers	Divide bus clocks independently; peripherals can run slower than CPU.
Run-time reconfiguration	Change HCLK prescaler on-the-fly for burst vs idle periods.

5 | Peripheral Power Strategies

Peripheral	Technique	Saving
GPIO	Set unused pins to analog mode (no pull, no Schmitt trigger).	~100 μ A per floating pin avoided
ADC	Use single-shot + power-down between samples.	~1–2 mA while active
UART	Use LPUART1 in Stop mode for wake-on-byte.	Full USART off saves ~500 μ A
SPI / I2C	Power-gate external sensor via GPIO-controlled VCC rail.	Sensor current eliminated
Timers	Use RTC/LPTIM instead of general-purpose timers in sleep.	~200 μ A saved
USB	Disconnect D+ pull-up when bus not needed.	~3–5 mA saved
Radio (BLE)	Use connection intervals ≥ 100 ms; TX power minimum viable.	Dominant power consumer
Display	Reduce refresh rate; use partial update; sleep when blanked.	10–50 mA saved

6 | RTC & Low-Power Wakeup Timer

The RTC runs from LSE (32.768 kHz crystal, ~0.8 μ A) or LSI (~1–2 μ A) and stays alive through Stop/Standby. Use it as an alarm to wake periodically.

```
// Wake every 30 seconds from Stop 2
HAL_RTCEx_SetWakeupTimer_IT(&hrtc, 30, RTC_WAKEUPCLOCK CK_SPRE_16BITS);
HAL_PWREx_EnterSTOP2Mode(PWR_STOPENTRY_WFI);
```

LSE vs LSI	LSE (crystal) more accurate; LSI (~ $\pm 5\%$) drifts but needs no crystal.
Wakeup counter	16-bit auto-reload; resolution selectable (1 s steps or sub-second).
Backup domain	RTC + backup registers powered by VBAT; survives main VDD loss.
Tamper / timestamp	Log power-loss events; useful for field diagnostics.

7 | FreeRTOS Tickless Idle Mode

By default FreeRTOS generates a tick interrupt every 1 ms, waking the CPU even when all tasks are blocked. **Tickless idle** suppresses ticks during idle periods and uses a low-power timer to resume at the next scheduled wakeup.

Enable	Set configUSE_TICKLESS_IDLE 1 in FreeRTOSConfig.h.
Built-in port	Cortex-M port uses SysTick + WFI automatically.
Custom hook	Implement vPortSuppressTicksAndSleep() for Stop 2 / deeper modes.
LPTIM tickless	STM32 HAL provides LPTIM-based tickless port for Stop-mode sleep.
Time correction	Scheduler updates tick count based on elapsed timer ticks on wakeup.
Minimum idle ticks	configEXPECTED_IDLE_TIME_BEFORE_SLEEP (default 2) — skip sleep if fewer than N ticks idle.

```
// Minimal custom tickless hook skeleton
void vPortSuppressTicksAndSleep(TickType_t xExpectedIdleTime) {
// 1. Program LPTIM for xExpectedIdleTime ticks
// 2. Enter Stop 2 via HAL_PWREx_EnterSTOP2Mode()
// 3. On wakeup: reconfigure clocks, correct tick count
}
```

8 | Measuring & Profiling Power Consumption

Tool / Method	Resolution	Use-case
Series ammeter (µA bench supply)	µA	Quick average current check.
Current shunt + oscilloscope	µA, µs waveform	Capture transient spikes during TX bursts.
Nordic PPK2 (Power Profiler Kit)	200 nA, 100 kSa/s	Fine-grained timeline of sleep/active cycles.
STM32 Current Consumption calc.	Estimate only	ST spreadsheet tool for budget planning.
Otii Arc	1 µA, voltage + current	Battery emulator + logger for field profiles.
GPIO toggling	—	Toggle pin at sleep entry/exit; view on LA or scope.

Measure average current over a full duty cycle (sleep + active + TX). Multiply by battery capacity (mAh) to estimate runtime: $T = C / I_{avg}$.

9 | Hardware Design for Low Power

LDO vs DCDC	DCDC converters are 85–95% efficient vs 40–70% for LDO at low loads.
Load switch	Use P-FET or dedicated load switch IC to cut power to sensor rail.
Pull resistors	Replace 10 kΩ pull-ups with 100 kΩ–1 MΩ on slow signals.
Bypass capacitors	Place 100 nF + 10 µF close to VDD pins to reduce noise and spikes.

PCB leakage	Use guarded traces and low-leakage diodes near uA-level nodes.
Oscillator choice	TCXO for accuracy; watch-crystal for ultra-low power standby.
Shutdown pin	Add hardware shutdown pin to sensors/radios for guaranteed power-off.
Battery fuel gauge	Use dedicated IC (e.g. MAX17048) for accurate SoC on LiPo cells.

10 | Common Pitfalls & Debug Checklist

Floating GPIO pins	Set all unused pins to analog/output-low before sleep.
Debug interface current	SWD/JTAG pins draw current; disable debug peripheral in production.
Regulator quiescent	LDO Iq at no-load can exceed target sleep current — check datasheet.
Peripheral not disabled	Even if clock-gated, a running peripheral can draw supply current.
DMA left running	DMA channels have their own enable bits; disable after transfer.
External pull-ups to Vcc	I2C 4.7 kΩ pull-up draws 700 μA continuously — switch off via GPIO.
Clocks not restored	After Stop wakeup, AHB/APB peripherals may be on MSI not PLL.
VBAT not connected	Float VBAT → RTC unreliable; tie to VDD or battery if RTC needed.

7-Day Course Roadmap:

Day	Topic
Day 1 ✓	Introduction — Architecture, GPIO, Memory, Protocols
Day 2 ✓	Interrupts, Timers & PWM
Day 3 ✓	UART, SPI, I2C — Deep Dive
Day 4 ✓	ADC/DAC & Sensor Interfacing
Day 5 ✓	RTOS Fundamentals (FreeRTOS)
Day 6 ✓	Power Management & Low-Power Design
Day 7	Bootloaders, OTA & System Design Project